

SORBONNE UNIVERSITÉ
CONCEPTION ET PRATIQUE DE L'ALGORITHMIQUE

ALGORITHMIQUE D'ESSAIS

Rapport de Projet SMA vs. Actor Model

Étudiant : CHEN Florent **N° étudiant :** 21101813

Enseignant : Binh-Minh Bui-Xuan

Année universitaire 2025–2026

Date de remise : 03 Mars 2026

0 Contents

1	Introduction	2
1.1	Contexte	2
1.2	Modélisation du problème	2
2	Structure de données	2
2.1	Tableaux de suivi	2
2.2	Classes internes clés	3
3	Analyse théorique des algorithmes	3
3.1	Contexte : SMA et modèle d'acteur	3
3.2	Algorithmes d'essaim pertinents	3
3.3	Protocole de communication	3
4	Présentation explicite des algorithmes implémentés	4
4.1	Architecture globale	4
4.2	PatientHunterBase : le cerveau commun	4
4.3	PatientHunterMain = TeamAMainBotChenFallavierTang : le tireur	6
4.4	PatientHunterSecondary = TeamASecondaryBotChenFallavierTang : l'éclaireur	6
5	Appréciation, améliorations et critique des algorithmes existants	6
5.1	Points forts de l'approche	6
5.2	Limites et pistes d'amélioration	7
6	Méthode d'obtention des données et test de performance	7
6.1	Protocole expérimental	7
6.2	Résultats observés	8
6.3	Discussion sur les algorithmes de contrôle	9
7	Discussion : SMA vs. Actor Model dans Simovies	9
7.1	Correspondance avec le modèle d'acteur	9
7.2	Émergence collective	9
7.3	Limites du modèle d'acteur dans ce contexte	9
8	Conclusion et perspectives	9

1 Introduction

1.1 Contexte

Le simulateur Simovies met en place une confrontation entre deux équipes de robots autonomes dans une arène de 3000 sur 2000 unités. Pour chaque équipe, elle est composée de cinq agents : trois robots principaux, qui sont des tireurs résistants mais lents et dotés d'un radar à courte portée, ainsi que deux robots secondaires, des éclaireurs rapides avec un radar longue portée, mais fragiles et incapables de tirer. L'objectif est d'éliminer tous les robots adverses en les détruisant par des tirs.

1.2 Modélisation du problème

Du point de vue des systèmes multi-agents (SMA), chaque robot est un agent autonome qui est capable d'analyser son environnement via à ces capteurs (comme le radar ou le capteur frontal) et il prend ses propres décisions (déplacement, rotation, tir) tout en communiquant avec ses alliés. Il n'y a aucun cerveau central : chaque robot décide seul, en temps réel, à partir de ce qu'il perçoit et de ce que ses coéquipiers lui ont signalé.

Le problème se décompose en quatre grandes axes :

1. **Perception** : mettre en commun les informations du radar avec les messages broadcast des alliés pour construire une carte partagée des ennemis.
2. **Navigation** : se déplacer de manière efficace sans se bloquer contre les murs ou les autres robots, et de maintenir une position tactiquement avantageuse.
3. **Action** : quand tirer, sur qui, et comment être certain de ne pas toucher un allié au passage ?
4. **Complémentarité des rôles** : tirer parti des capacités de chaque agent. Le Secondary, avec son radar longue portée, il permet de détecter des ennemis bien au-delà de ce que le Main peut voir. Il transmet ces positions par un broadcast, ce qui permet au Main de tirer à pleine portée sur des cibles qu'il n'aurait jamais repérées seul. Ainsi le duo forme un pont entre détection et destruction.

2 Structure de données

La classe abstraite `PatientHunterBase` centralise toutes les structures partagées entre les sous-classes `PatientHunterMain = TeamAMainBotChenFallavierTang` et `PatientHunterSecondary = TeamASecondaryBotChenFallavierTang`.

2.1 Tableaux de suivi

`listeEnnemis (ArrayList<PointSuivi>)`

Positions estimées des ennemis, mises à jour par radar direct ou broadcast allié. Chaque point mémorise sa source (RADAR ou DIFFUSION), son horodatage (`dernierStepMiseAJour`), et un flag `predict` indiquant s'il s'agit d'une position prédite par les projectiles.

`listeProjectiles (ArrayList<PointSuivi>)`

Positions des projectiles ennemis détectés. Lorsqu'une balle est suivie sur deux steps consécutifs, alors sa trajectoire permet d'identifier la position du tireur.

`pistesBalles (ArrayList<PisteBalle>)`

Pistes de projectiles : chaque `PisteBalle` conserve les deux dernières positions d'une balle, permettant de calculer son vecteur vitesse et donc la direction d'origine du tir.

positionsAllies (ArrayList<PositionAllie>)

La dernière position connue de chaque allié, diffusée régulièrement. Sert principalement à vérifier qu'on ne va pas tirer sur un allié.

indicesTirRecents (ArrayList<IndiceTirRecent>)

Les tirs récents signalés par les alliés : position, direction, horodatage. Permettent d'identifier les balles alliées dans le radar.

2.2 Classes internes clés

Classe	Rôle
PointSuivi	Point suivi (ennemi ou projectile) avec source, horodatage et flag de prédiction
PisteBalle	Suivi d'un projectile sur deux steps pour déduire la vitesse
PositionAllie	Dernière position connue d'un allié
IndiceTirRecent	Mémorisation d'un tir allié récent pour filtrer les balles radar
CandidatCible	Cible sélectionnée avec niveau de priorité (0 à 4)
PlanDepart	Cap et nombre de steps pour la phase de départ

3 Analyse théorique des algorithmes

3.1 Contexte : SMA et modèle d'acteur

Un **Système Multi-Agents** (SMA) est un ensemble d'agents autonomes qui interagissent dans un environnement partagé. Chaque agent se base sur sa propre vision du terrain pour décider et agir de manière autonome. L'intelligence du groupe n'est pas programmée à l'avance, elle émerge naturellement de la manière dont les agents interagissent entre eux [3].

Le **modèle d'acteur** (Actor Model), formalisé par Hewitt en 1973, décrit un calcul concurrent où tout acteur peut : (1) envoyer des messages à d'autres acteurs, (2) créer de nouveaux acteurs, (3) définir son comportement pour le prochain message reçu. Dans Simovies, chaque robot est un acteur : il dispose d'une boîte aux lettres (les messages broadcast), traite tous les messages reçus à chaque itération (le step), et émet des actions.

3.2 Algorithmes d'essaim pertinents

Dans la littérature sur l'**intelligence en essaim** [1], deux grandes familles d'algorithmes sont mobilisées :

Comportements réactifs (subsumption). L'architecture de Brooks organise les comportements en niveaux de priorité. L'évitement d'un obstacle devient prioritaire et prendre le dessus temporairement l'ordre de se rapprocher d'un ennemi. **PatientHunter** utilise ce principe avec une pile de priorités explicite dans chaque **step()**.

Stigmergie et mémoire partagée. Les fourmis utilisent des phéromones pour coordonner sans communication directe. Dans [2], les auteurs appliquent l'optimisation par colonie de fourmis dans un modèle d'acteur distribué. Dans **PatientHunter**, la table **listeEnnemis** joue le rôle d'une phéromone numérique : chaque détection laisse une trace qui s'évapore progressivement (TTL).

3.3 Protocole de communication

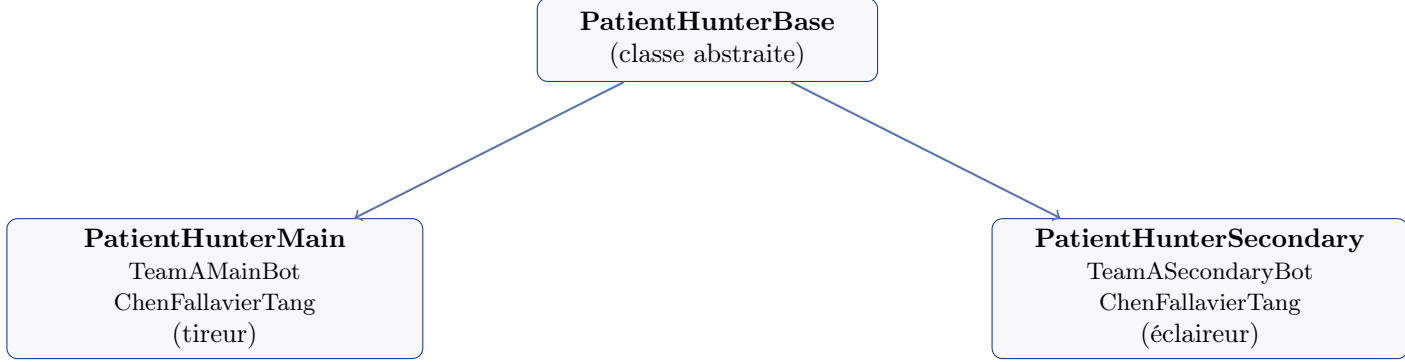
Le protocole de message est compact et sans état partagé centralisé :

PHR|<TYPE>|<X>|<Y>|<DIR_OU_NA>|<ID_EMETTEUR>

Les types sont ENEMY, ENEMY_PREDICTED, BULLET, FIRE, POS. Chaque message expire après un TTL (entre 10 et 40 steps selon le type), ce qui simule l'évaporation des phéromones.

4 Présentation explicite des algorithmes implémentés

4.1 Architecture globale



4.2 PatientHunterBase : le cerveau commun

PatientHunterBase est la classe abstraite dont héritent les deux robots. Elle concentre toute la logique partagée : initialisation, odométrie, radar, communication entre alliés, suivi des ennemis/projectiles et gestion du déblocage.

Initialisation

Au début de la partie, la méthode **activerBase()** s'occupe de tout remettre à zéro dans le robot : les compteurs sont réinitialisés, les mémoires vidées, et son état interne préparé. Une fois que c'est fait, c'est au tour de **assignerPositionDepart()** de prendre le relais. Cette méthode place l'unité sur la carte en fonction de son équipe et de son rôle, tout en réglant sa vitesse et son orientation de départ. Enfin, **construirePlanDepart()** prépare le plan de départ du tireur, en définissant le nombre de ticks du démarrage et le cap de séparation initial.

Cycle de vie commun

À chaque tour, **stepBase()** qui gère tout ce qui concerne la mise à jour de l'état du robot : recalage de la position, traitement des messages alliés et analyse radar (ennemis et projectiles). Cette fonction retourne ensuite les détections immédiates du tour. Pour garantir la mobilité, **verifierBlocageEtMettreAJourDeblocage()** vérifie s'il est coincé ; si une collision se produit sans que les capteurs ne l'aient détectée, elle déclenche une routine spéciale pour le décoincer.

Suivi des projectiles et inférence du tireur

assignerAPiste() associe chaque balle radar à une piste déjà connue ou en crée une nouvelle. Lorsqu'une piste dispose de deux positions consécutives, **estimerDirectionTirPourPiste()** calcule le vecteur vitesse $\vec{v} = (x_2 - x_1, y_2 - y_1)$ et en déduit la direction opposée $\alpha = \text{atan2}(-v_y, -v_x)$ pointant vers le tireur estimé :

$$(x_e, y_e) = (\text{borner}(x_{\text{moi}} + d \cos \alpha, 0, 3000), \text{borner}(y_{\text{moi}} + d \sin \alpha, 0, 2000))$$

avec $d = 500$ unités. **classerTireurInfere()** décide ensuite si ce tireur ressemble à un allié, à un ennemi connu, ou si l'information reste incertaine. **nettoyerPistesBallesExpirees()** supprime les pistes trop anciennes, et **synchroniserListeProjectilesDepuisPistes()** reconstruit la liste des projectiles à partir des pistes encore valides.

Filtrage des balles alliées

Trois méthodes évitent de traiter les balles alliées comme des contacts ennemis. `estProbablementMaPropeBalle()` permet d’abord d’ignorer ses propres tirs `estProbablementTirDansDirectionAllie()` filtre les balles qui semblent venir d’un allié. `estProbablementBalleAllie()` compare une balle radar avec les derniers messages `FIRE` reçus pour voir si elle correspond à un tir allié connu.

Mémoire des ennemis et des projectiles

`mettreAJourEnnemI()` fusionne ou ajoute une position d’ennemi dans la mémoire du robot par moyenne pondérée, en donnant plus de priorité aux données du radar qu’aux messages reçus. `mettreAJourProjectile()` fait pareil pour les projectiles. `doitIgnorerEntrant()` empêche qu’une information soit écrasée par une observation d’un radar ou broadcaste moins fiable. `nettoyerPointsDiffusionPerimes()` supprime les informations trop anciennes dans les listes d’ennemis, de projectiles et d’indices de tir, tandis que `conserverMemoireEnnemLointain()` permet de conserver temporairement en mémoire un ennemi très éloigné, afin de ne pas l’oublier trop rapidement.

Sélection et verrouillage de la cible

`mettreAJourVerrouEnnemDepuisRadar()` verrouille l’ennemi pour éviter de changer de cible trop souvent (quand il y a plusieurs ennemi). `selectionnerCibleParPriorite()` choisit alors la meilleure cible selon le protocole appliqué : les ennemis détectés directement par le radar, l’ennemi verrouillé récent, l’ennemi radar mémorisé, les informations partagées par ses alliés, l’ennemi prédit par trajectoire de balle, le projectile radar, le projectile diffusé. `appliquerVerrouCible()` renforce cette stabilité en lissant la position de l’adversaire sur plusieurs tours pour éviter qu’elle change sans arrêt. `definirVerrouCible()` enregistre une nouvelle cible verrouillée. `selectionnerEnnemLePlusProche()`, `selectionnerEnnemPreditLePlusProche()` et `selectionnerLePlusProche()` sont les méthodes utilitaires qui parcourent les listes pour retourner le point le plus proche selon la source voulue.

Navigation et déblocage

`seDeplacerVers()` gère le déplacement principal vers une cible en gérant l’alignement angulaire, l’évitement des obstacles, de la réorientation et du déblocage. `executerDeblocageDroite()` applique la méthode de déblocage en donnant la priorité à la droite, en alternant entre des marches arrière et des rotations en suivant la constante `PERIODE_ZIGZAG_DEBLOCAGE`. `avancerAvecOdometrie()` avance tout en mettant à jour la position estimée localement.

Anti-tir fratricide (triple garde)

Avant de tir, il fait trois vérifications. `allieDevant()` regarde si un allié se trouve devant le canon via le capteur frontal. `allieDansLigneDeViseeDepuisListe()` vérifie si un allié connu se trouve dans l’axe de tir, via le calcul de l’angle de blocage géométrique $\theta_{bloc} = \max(\frac{\pi}{20}, \arcsin(\frac{r_{allie}}{d_{allie}}))$. `allieDansLigneDeViseeDepuisRadar()` fait la même vérification avec les alliés vus directement au radar, avec une marge de sécurité supplémentaire.

Communication

`traiterMessagesEntrants()` lit les messages qui arrivent sur le réseau et les transforme en infos exploitables : la positions des ennemis, les tirs, les positions alliées. `tirerAvecIndice()` tire et envoie simultanément un message `FIRE` aux alliés, permettant à `ajouterIndiceTirRecent()` peut enregistrer ce tir pour ne pas le prendre en compte plus tard et éviter les fausses alertes. `diffuserDetection()` envoie un message court sur le bus d’équipe au format `PHR|TYPE|X|Y|DIR|ID`. `mettreAJourPositionAllie()` met à jour la position d’un allié dans la mémoire partagée.

4.3 PatientHunterMain = TeamAMainBotChenFallavierTang : le tireur

`activate()` lance le programme d'initialisation du Main et récupère le plan de départ. À chaque tour, `step()` s'occupe de mettre à jour la perception avec `stepBase()`. Si la phase de départ est toujours en cours, elle est appliquée. Puis le système choisit une cible, il essaye de tirer et si ça marche pas alors il se déplace vers la cible ou se débloque s'il est bloqué.

Phase de départ. `executerStrategieDepart()` gère le démarrage du robot Main pendant 380 steps. Les robots 1 et 3 se séparent selon leur identifiant et leur côté : l'un pivote à gauche pendant 25 steps, avance 330 steps, puis se remet tout droit ; l'autre fait le symétrique. Cela crée une formation en éventail qui couvre l'axe de progression dès le début de la partie.

Sélection de la cible et condition de tir. `selectionnerPointTirFraisDepuisCible()` le robot s'assure de ne viser que des cibles dont la position a été actualisée il y a moins de 6 tours (`FIRE_TARGET_MAX_AGE_STEPS`), évitant de tirer sur une position obsolète. `calculerAngleStable()` donne l'angle de tir que `tentativeTir()` va utiliser. La méthode gère d'abord les cas où dx ou dy sont nuls, puis renvoie un l'angle normalisé. `tentativeTir()` vérifie ensuite si le tir est possible et sûr : il le bloque si un allié est devant, si l'axe de tir est dangereux selon la triple garde, ou si une épave est à moins de 450 unités devant le canon. Dans ce dernier cas, le robot recule puis tourne à droite pour se dégager.

4.4 PatientHunterSecondary = TeamASecondaryBotChenFallavierTang : l'éclaireur

`activate()` lance le programme du Secondary, qui mémorise sa santé initiale de l'unité et définit son comportement de départ sur l'état `ETAT_AVANCE`. À chaque tour, `step()` vérifie dans l'ordre : si les points de vie sont trop bas alors le robot prend la fuite. Si un mur est actif, il tente de le débloquer. Et enfin, il adopte le comportement normal correspondant à la phase en cours

Phase 1 (< 4000 steps) : lignes droites. Pour savoir à quelle distance est l'ennemi le plus proche, `trouverEnnemiLePlusProche()` regarde d'abord ce que le radar voit directement ce tour-ci, et si rien n'est en vue, il fouille la mémoire partagée. `trouverDistanceEnnemiLePlusProche()` en déduit la distance. Si un ennemi est à moins de 480 unités le robot recule ; si la voie est libre il avance en ligne droite ; si bloqué il recule et laisse la Base déclencher le déblocage.

Phase 2 (≥ 4000 steps) : orbite.

- **Trop loin** ($d > 720$) : `seDeplacerVers()` rapproche le robot de l'ennemi.
- **Trop près** ($d < 480$) : fuite vers le point opposé (`borner( $x_{moi} - dx$ ), borer( $y_{moi} - dy$ )`).
- **Zone morte** ($480 \leq d \leq 720$) : orbite latérale à $\pm \frac{\pi}{2}$ de la direction ennemi, avec signe alternant selon `idInstance` pour que les deux éclaireurs orbitent en sens opposés.

Fuite critique. `doitFuir()` détecte si la santé est passée sous 20% de la valeur initiale. `fuirAvecPeuDeVie()` oriente alors le robot à l'opposé de l'ennemi le plus proche, ou vers le bord allié si aucun ennemi n'est visible.

5 Appréciation, améliorations et critique des algorithmes existants

5.1 Points forts de l'approche

Le fait de tout centraliser dans `PatientHunterBase` s'est avéré être un bon choix : odométrie, communication, suivi de cibles et gestion du tir fratricide sont écrits une seule fois et partagés par les deux robots, ce qui évite les bugs de duplication et facilite les modifications. La table de suivi

avec TTL imite efficacement l'évaporation de phéromone décrite dans [2], donnant au système une mémoire collective sans état centralisé.

Le triple filtrage du tir fratricide (capteur frontal + liste + radar) est plus robuste que les implémentations classiques qui n'utilisent qu'un seul mécanisme. Si l'une des trois détecte un allié dans l'axe, le tir est bloqué, ce qui réduit presque à zéro le risque de toucher un coéquipier.

5.2 Limites et pistes d'amélioration

Cohérence de l'odométrie. L'odométrie actuelle est volontairement conservatrice : la position estimée n'est mise à jour qu'après l'ordre de mouvement, et seulement si le capteur frontal indiquait un passage libre au moment de l'action. Cela limite la dérive quand le robot frotte un obstacle ou un allié, sans dépendre d'une API interne.

Fiabilité du filtrage des balles alliées. On a mis en place un système de reconnaissance des tirs pour distinguer les balles alliées des balles ennemies, ce qui permet au robot de tirer vers la position prédite d'un ennemi sans se laisser perturber par les tirs de ses coéquipiers. Sauf que des fois, ça coince. Quand plusieurs robots canardent en même temps ou qu'ils sont trop proches, une balle alliée peut être prise pour une balle ennemie.. Le robot réagit alors à un faux contact, ce qui désorganise sa prise de décision et peut l'amener à viser

une position inexistante. On pourrait améliorer la détection en rendant plus stricte la règle qui associe un tir allié déjà enregistré à la balle qu'on voit arriver. Par exemple, en réduisant la marge d'erreur sur la position estimée de la balle.

Esquive des balles ennemies. Puisque le robot maintient déjà une liste des projectiles ennemis repérés dans `listeProjectiles`, il serait naturel d'exploiter cette information pour esquiver les tirs. L'idée serait de détecter qu'une balle ennemie arrive dans notre axe et de réagir par un mouvement vertical (monter ou descendre), plutôt que de se déplacer latéralement. Un déplacement gauche-droite est en effet trop lent compte tenu de la vitesse des projectiles : le temps de pivoter et d'avancer dans la nouvelle direction, la balle est déjà passée. Un pas vertical rapide serait plus efficace pour sortir de la trajectoire.

Coordination inter-rôles. Actuellement, les éclaireurs et les tireurs n'ont pas de protocole explicite de *désignation de cible* : chacun choisit indépendamment. Un mécanisme de *token* ou d'enchère (inspiré du modèle d'acteur de [4]) permettrait de concentrer les tirs sur un seul ennemi et de l'éliminer plus vite.

Prédiction de mouvement. La position inférée d'un ennemi est supposée statique. Un filtre de Kalman simple donnerait une prédiction de position à $t + k$ steps, augmentant le taux de tirs effectifs.

Phase de départ. La logique de *la formation en éventail* est codée en dur avec des seuils de ticks fixes. Elle pourrait être remplacée par une formation adaptative qui tient compte de la stratégie des ennemis.

6 Méthode d'obtention des données et test de performance

6.1 Protocole expérimental

Les données de performance ont été obtenues en faisant s'affronter **PatientHunter** contre l'implémentation de référence fournie avec le simulateur. Les runs ont été répétés plusieurs fois pour chaque configuration afin de lisser les aléas du placement et des collisions initiales.

Les métriques collectées sont :

- nombre de robots ennemis immobilisés en fin de partie,
- nombre de robots alliés perdus,
- durée (en steps) avant la première neutralisation ennemie,
- taux de tirs fratricides (tirs ayant touché un allié).

6.2 Résultats observés

Les tests ont été menés sur deux durées : sans limite de temps et avec une limite de 5 minutes par partie. Chaque configuration a été jouée une fois par côté lorsque c'était possible.

Sans limite de temps

Adversaire	Pertes PatientHunter	Éliminations PatientHunter
RandomFire (côté A)	2/5	5/5
RandomFire (côté B)	1/5	5/5
Fugitive (côté A)	3/5	4/5
Fugitive (côté B)	2/5	2/5
Berzerk (côté A)	3/5	5/5
Berzerk (côté B)	3/5	4/5
CampFire/CampBot (côté A uniquement)	5/5	2/5
PatientHunter vs. PatientHunter	3/5	4/5

Avec limite de 5 minutes

Adversaire	Pertes PatientHunter	Éliminations PatientHunter
RandomFire (côté A)	2/5	4/5
RandomFire (côté B)	2/5	4/5
Fugitive (côté A)	3/5	5/5
Fugitive (côté B)	2/5	5/5
Berzerk (côté A)	1/5	5/5
Berzerk (côté B)	4/5	3/5
CampFire/CampBot (côté A uniquement)	4/5	3/5
PatientHunter vs. PatientHunter	4/5	2/5

PatientHunter domine clairement les adversaires à comportement aléatoire ou erratique (RandomFire, Berzerk) avec un taux d'élimination de 4 à 5 robots sur 5 dans la quasi-totalité des cas. Les résultats contre Fugitive sont plus mitigés côté B (2/5 éliminations sans limite), ce qui suggère que cet adversaire adopte une stratégie d'évitement difficile à contrer quand **PatientHunter** part du mauvais côté.

Le cas CampFire/CampBot est le plus défavorable : **PatientHunter** perd tous ses robots (5/5) sans limite de temps, et n'en élimine que 2 à 3. Cette stratégie de campement statique met en défaut la logique de déplacement vers la cible, qui expose les robots à des tirs concentrés sur une position fixe.

Enfin, le miroir **PatientHunter** vs. **PatientHunter** donne des résultats légèrement asymétriques selon le côté et la durée, ce qui est attendu dans un système sans coordination centralisée.

6.3 Discussion sur les algorithmes de contrôle

La stratégie de l'éclaireur en phase 2 (orbite) maximise la visibilité radar continue sans exposer le robot : il reste à portée de radar long tout en étant hors de portée de tir des robots principaux adverses. Cela permet aux tireurs alliés de recevoir des positions ennemies fraîches par broadcast, améliorant la précision des tirs.

Le mode de déblocage (`modeDeblocageDroite`) à 100 steps maximum a montré quelques cas de boucle : un robot peut tourner longtemps dans un coin. Une amélioration serait de randomiser partiellement la direction de déblocage.

7 Discussion : SMA vs. Actor Model dans Simovies

7.1 Correspondance avec le modèle d'acteur

Simovies est un exemple canonique d'Actor Model appliqué aux SMA [3]. Chaque robot est un acteur avec :

- une **boîte aux lettres** (le bus de broadcast),
- un **état interne** encapsulé (tables de suivi, position, santé),
- un **comportement** défini par `step()`, exécuté à chaque message implicite de l'horloge du simulateur.

La principale différence avec un modèle d'acteur pur (Erlang, Akka) est que les messages sont ici *broadcast non fiable* sans garantie de livraison ni d'ordre, ce qui oblige à concevoir des algorithmes robustes à la perte de messages (d'où les TTL et les sources multiples).

7.2 Émergence collective

Aucun robot de PatientHunter ne possède de vue globale. Pourtant, la combinaison de broadcasts de position (tous les 3 steps) et de la table `listeEnnemis` fait émerger une *carte collective* de l'ennemi. Ce mécanisme est proche de la stigmergie numérique décrite dans [2] pour les colonies de fourmis : les informations sont déposées dans une mémoire partagée et s'évaporent naturellement par TTL.

7.3 Limites du modèle d'acteur dans ce contexte

Le modèle d'acteur pur suppose que chaque acteur traite *un* message à la fois. Dans Simovies, le step est synchrone pour toute l'équipe : il n'y a pas de concurrence réelle, ce qui simplifie la coordination mais limite le passage à l'échelle. Pour un essaim de 100 robots, un modèle asynchrone (type ROS) serait nécessaire.

8 Conclusion et perspectives

Le projet PatientHunter démontre que peu d'éléments sont nécessaires pour obtenir un fonctionnement efficace d'un essaim d'agents. Avec des comportements réactifs bien organisés, une mémoire collective qui expire après un certain temps et une communication par diffusion, on obtient un comportement collectif efficace, même dans un contexte de compétition. Les idées de base des algorithmes d'essaim, comme la stigmergie, l'émergence et la décentralisation, se retrouvent naturellement dans le code du projet.

Pour ce qui est des améliorations, plusieurs pistes méritent d'être explorées. On pourrait par exemple tenter de prédire la position de l'adversaire en utilisant un filtre de Kalman, mieux organiser les tirs avec un protocole de désignation de cibles, ou encore adapter la formation de l'équipe au fil de la partie.

D'une manière plus générale, les systèmes multi-agents et le modèle d'acteur vont très bien ensemble. Les SMA décrivent comment les agents se comportent, leurs rôles, leurs interactions et

leur environnement. Quant au modèle d'acteur, il sert de base pour exécuter plusieurs agents en parallèle tout en prenant en compte les pannes éventuelles. Combiner les deux devient d'ailleurs de plus en plus courant, comme le montrent Simovics ou ROS [4], pour les systèmes robotiques distribués.

8 References

- [1] A. Chakraborty and A.K. Kar, *Swarm Intelligence: A Review of Algorithms*, Nature-Inspired Computing and Optimization, Springer, 2017.
- [2] M. Starzec, G. Starzec, A. Byrski and W. Turek, *Distributed ant colony optimization based on actor model*, Journal of Computational Science, 2019.
- [3] W. van der Hoek and M. Wooldridge, *Multi-Agent Systems*, chapter 24 in Handbook of Knowledge Representation, Elsevier, 2007.
- [4] R.C. Cardoso and A. Ferrando, *A Review of Agent-Based Programming for Multi-Agent Systems*, Computing, Springer, 2021.